

# KAVACH-AI 6-Day Execution Plan

## KAVACH-AI — 6-Day SIH Execution Plan

### Sovereign On-Premise Agentic AI Workbench (SIH26117, MRPL)

One story to hold onto through all 6 days: > An engineer gives the workbench a real industrial task. The system understands the input, chooses the right local AI, retrieves the correct evidence, does the work, verifies the result, generates the document, and proves the entire process stayed inside the organization.

Everything below either builds that story or it gets cut.

### Day 0 — Before the clock starts (mandatory, not optional)

**Hardware freeze — distributed across 3 machines** (2× 6GB RTX 4050 + 1 MacBook), since a single 6GB card cannot hold a text model and a vision model at once without OOM crashes:

| Machine                  | Role             | Load                                                                                   |
|--------------------------|------------------|----------------------------------------------------------------------------------------|
| Laptop 1 (RTX 4050, 6GB) | The Brain        | Ollama + Llama-3.1-8B-Instruct, 4-bit GGUF (~5.5GB) — general reasoning + routing only |
| Laptop 2 (RTX 4050, 6GB) | The Engine       | Ollama + Qwen2.5-Coder-7B and Qwen2.5-VL-7B (both 4-bit), hot-swapped by task          |
| MacBook                  | The Orchestrator | Next.js/React frontend, FastAPI backend, vector DB (Qdrant/FAISS), Docker sandbox      |

Network the three machines via a **local offline switch** (not a mobile hotspot — a physical switch can be dramatically unplugged mid-demo; a phone’s Wi-Fi toggle is a weaker visual proof) or a hotspot with mobile data off if a switch isn’t available.

⚠ **Verify on Day 0, don’t assume:** Qwen2.5-VL-7B needs meaningfully more VRAM than the text-only Coder-7B on the same card, because of the vision encoder and image-token overhead — community guidance points to 8GB+ cards for this model, not 6GB. **Test Laptop 2 with an actual P&ID image (not just a text prompt) on Day 0.** If it OOMs, have a smaller vision model (2B-class VLM) ready as a fallback so this doesn’t get discovered mid-hackathon.

**Model hot-swap latency is real** — swapping Coder-7B ↔ VL-7B on Laptop 2 takes real seconds, sometimes tens of seconds. Don’t hide this in the demo: show a visible “Switching to vision model...” UI state. A silent pause looks like a crash; a labeled one looks like a system that manages its own resources.

**Install everything in advance** (no downloads during the event): - Backend: Python, FastAPI, Uvicorn, Pydantic - Frontend: Node, React, Tailwind, Framer Motion - AI: Ollama or vLLM - Documents: PyMuPDF, python-docx, openpyxl, pandas - OCR: Tesseract or PaddleOCR - Storage: SQLite/PostgreSQL, FAISS or Qdrant

**Download models in advance:** - General: Llama 3.1 8B (or similar) - Vision: Qwen2.5-VL 7B - Code: Qwen2.5-Coder 7B - Embedding: BGE or E5

**Prepare demo\_data/ folder** — this is not optional, build it Day 0: inspection\_report.pdf, scanned\_report.pdf, sample\_sop.pdf, maintenance\_history.csv, equipment\_image.png, sample\_pid.png, previous\_incident.pdf, report\_template.docx, and one document containing a stale SOP pair (Rev-2 vs Rev-4) and one document with embedded prompt-injection text.

**Exit criteria:** offline startup works, every model loads once with internet off, and the MacBook can call both laptops’ Ollama instances over the local network.

### Day 1 — Foundation: a website that talks to a local model

**Goal:** browser → file upload → backend → local AI → response. Nothing agentic yet.

- Frontend shell: sidebar (Dashboard, Documents, AI Jobs, Knowledge, Artifacts, Audit, Network, Settings), upload box,

request box, “Start task” button.

- Backend endpoints: `POST /upload`, `POST /task`, `GET /job/{id}`, `GET /artifact/{id}`, `GET /audit/{id}`, `GET /health`.
- File save path: `/workspace/jobs/<job_id>/input.pdf`, `metadata.json`, `output/`, `audit.json`.
- Connect FastAPI → Ollama → local LLM. Test “summarize this text” **with internet physically off**.
- Job states: `RECEIVED` → `PROCESSING` → `RETRIEVING` → `EXECUTING` → `VERIFYING` → `COMPLETED` / `FAILED`.

**Do NOT touch today:** GraphRAG, MCP, memory, multiple agents, 3D anything.

**Exit criteria:** upload a PDF, get a real local-model response, with the internet cable pulled out.

---

## Day 2 — Router + bounded agent

---

**Goal:** turn the chatbot into a task-aware agent.

- Model registry (JSON): `general` (summary/reasoning/report), `vision` (image/scan/pid), `coder` (python/calculation).
- Router — **rule-based, not learned**: `PDF+summary` → `general` · `image/P&ID` → `vision` · `CSV+calculation` → `coder` · `scanned PDF` → `vision+OCR`.
- Agent state machine: `INTAKE` → `PLAN` → `RETRIEVE` → `ACT` → `OBSERVE` → `VERIFY` → `COMPLETE`, with `VERIFY: FAIL` → `REFLECT` → `VERIFY` (max\_steps=8, max\_retries=2).
- UI trace — show a **concise operational trace**, never hidden chain-of-thought: “Request understood · Vision model selected · Report analyzed · SOP retrieved · Evidence verified · Report generated.”

**Exit criteria:** router visibly picks a different model for two different task types; agent stops itself at the step/retry limit instead of looping forever.

---

## Day 3 — RAG + multimodal understanding

---

**Goal:** the system actually reads MRPL’s documents correctly.

- Digital PDF path: text extraction → chunking → embedding → vector DB.
- Scanned PDF path: OCR → page image → vision model.
- P&ID path: image → vision model → question answering.
- Simple RAG first (no GraphRAG): query → embedding → top-5 chunks → LLM → answer **with sources** ( `document`, `page`, `revision`, `confidence` ).
- CSV support: pandas-based filtering (e.g. “previous failures for Pump P101”).

**Exit criteria (this is the line in the sand):** *scanned inspection report* → *evidence* → *grounded answer* must work end-to-end by the end of today. If it doesn’t, nothing after this matters — stop and fix this first tomorrow morning before adding anything new.

---

## Day 4 — Tools, sandbox, and the real deliverable

---

**Goal:** the agent does actual work and produces a real file.

- Minimum 6 tools: `read_document`, `search_knowledge`, `read_csv`, `run_python`, `generate_docx`, `human_approval`.
- Sandbox for code: Docker, no internet, read-only input, limited CPU/RAM/runtime.
- DOCX generator — the killer artifact. Sections: Purpose, Inspection Date, Key Findings, Evidence Table, Recommendations, Unresolved Items, Source References, Reviewer, and an explicit **“AI-generated draft — human approval required”** label.
- Human approval UI: `[Approve]` `[Modify]` `[Reject]` — the system never self-approves.

**Exit criteria:** *scanned inspection report* → *real, opens-without-error .docx file* works end-to-end.

---

## Day 5 — Sovereignty, security, audit (this is what separates you from every other “agentic AI” team)

---

- Network isolation: default-deny firewall between the workbench and the internet.
- **Live sovereignty monitor, built with psutil on the MacBook — not eBPF.** eBPF does not run natively on macOS (the Darwin/XNU kernel doesn’t support it without a Linux VM workaround), so it’s the wrong tool here. `psutil`-based socket/connection monitoring is pure Python, cross-platform, and gives the same proof — local services ✓, external calls 0, outbound traffic 0, sandbox network OFF.
- Audit log: every action logged as `{time, actor, event, tool, status}`.

- SHA-256 hashing of input and output files, shown in the UI.
- **Live security demo:** attempt an internet call in front of the judge → blocked, on screen, in real time.

**Exit criteria:** you can unplug the network cable mid-demo and the UI shows it, live, without narration.

## Day 6 — Integration, rehearsal, and the four demos

**No new features today.** This day is stabilization and rehearsal only.

**The four judge-visible demos that must exist:** 1. **Auto model routing** — text → general, image → vision, code → coder, shown live via a route card. 2. **Scanned report → approval note** — the full Day 3+4 pipeline, end to end. 3. **Coding in the sandbox** — a calculation task, verified, with visible pass/fail status. 4. **P&ID understanding** — upload a diagram, ask a bounded question, get a cited answer.

**Final “wow” layout:** split screen — left: agent activity trace, center: the generated DOCX, right: the live sovereignty panel. Judges see AI + work + security + proof simultaneously.

**Rehearsal checklist:** two full timed dry runs, one deliberately injected failure (to show the retry/abstain path), backup laptop/USB with a pre-generated fallback output in case live inference stalls.

## Team split (6 people)

| Person              | Owns                                                                 | Primarily works on                  |
|---------------------|----------------------------------------------------------------------|-------------------------------------|
| 1 — Runtime/Backend | FastAPI, agent state machine, job management, retries                | MacBook                             |
| 2 — AI/Models       | Ollama/vLLM, model registry, router, fallback                        | Laptop 1 + Laptop 2                 |
| 3 — RAG/Multimodal  | PDF, OCR, embeddings, vector DB, P&ID, evidence                      | MacBook (calls Laptop 2 for vision) |
| 4 — Tools/Sandbox   | Docker sandbox, Python execution, CSV, DOCX generation               | MacBook                             |
| 5 — Security        | Firewall, <code>psutil</code> network monitor, audit log, hashing    | MacBook                             |
| 6 — Frontend/UX     | Dashboard, agent timeline, evidence panel, route card, network panel | MacBook                             |

## Priority tiers

- **P0 (must-have, days 1-5 core):** web UI, local LLM, router, agent, RAG, OCR, vision, DOCX, verification, human approval, network proof, audit.
- **P1 (only if P0 is done early):** knowledge conflict detector, failure→recovery demo, prompt-injection defense.
- **P2 (cut without guilt):** AI Watchtower, evidence map graph visualization, GraphRAG, temporal knowledge graph, learned router, 3D refinery, sensor simulation.

## Appendix — Light features to remember (not committed, zero build-plan impact)

Two long differentiator lists were shared with ~29 ideas total. Most need real new engineering and are already parked in P2. The ones below are different — each one **reuses data or infrastructure that P0/P1 is already building**, so the marginal cost is small (mostly a UI display, not a new subsystem). Nobody has to build any of these. If a day finishes early, pick from here — don’t invent something new under time pressure.

| Idea                                                                                                     | Why it’s cheap                                                         | Reuses                                |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------|
| <b>Attack the AI</b> — buttons: “try external call”, “use stale SOP”, “unauthorized file”                | Just exposes checks you’re already building as clickable demo triggers | Day 5 security + Day 3 revision check |
| <b>AI Red Team Score</b> — one summary score (evidence/policy/revision/artifact/security) after each job | Pure aggregation of pass/fail flags you already compute                | Verifier pipeline (Day 3-4)           |

|                                                                                                                                                                                                    |                                                                                        |                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------|
| <b>Local data lineage</b> — “this .docx came from these 3 files, these 2 models, this job ID”                                                                                                      | Audit log already has this; just render it near the download button                    | Audit log (Day 5)                 |
| <b>Model router on screen</b> — show the route card live, not buried in logs                                                                                                                       | It’s a UI panel over data the router already produces                                  | Router (Day 2)                    |
| <b>Mission card</b> — reframe the agent trace as a numbered plan with a completion count (“4/5 steps”)                                                                                             | Same trace data, different presentation                                                | Agent state machine (Day 2)       |
| <b>Safe fallback mode</b> — if the vision model is unavailable, drop to OCR-only and say so instead of failing                                                                                     | A second code path plus one honest UI message                                          | Router + OCR pipeline             |
| <b>Uncertainty budget</b> — show confidence as “8 known facts, 2 inferences, 1 unknown” instead of a bare percentage                                                                               | Same evidence objects, just counted and displayed differently                          | Evidence/verifier objects (Day 3) |
| <b>“What would’ve happened without KAVACH-AI”</b> — one slide, not code: manual steps/time vs. AI steps/time                                                                                       | Zero engineering — it’s a benchmark you measure once and put on a slide                | None (presentation only)          |
| <b>Shadow mode</b> — one line in the pitch: “before deployment, this can run alongside the real workflow and only compare, never act”                                                              | Zero engineering — it’s a deployment-story sentence, not a feature                     | None (presentation only)          |
| <b>Judge-verification export</b> ( <i>my addition</i> ) — one button that zips the audit log, file hashes, and generated artifact into a small bundle the judge can inspect offline after the demo | You already generate hashes and audit events; this just packages what exists           | Audit log + hashing (Day 5)       |
| <b>End-of-job summary card</b> ( <i>my addition</i> ) — total elapsed time, models used, verifier pass rate, shown once a job completes                                                            | Pure telemetry you’re already logging, formatted as one card instead of scattered logs | Job state + audit log             |

Skip entirely unless a day finishes very early: AI Watchtower, Evidence Map graph, Claim Graph, Human Correction Learning Loop, Counterfactual/“What Changed” engine, Cost-aware model selection with live GPU/ETA numbers, Digital Twin asset graph. Each of these needs a genuinely new subsystem, not a display on top of an existing one — that’s exactly the kind of scope creep the 6-day plan can’t absorb.